

SmartBank: Design and Implementation of a Secure SaaS-Style Digital Banking Platform

Capstone Project

Vignesh Sriram
Binghamton University
MS in Information Systems
B-Number: B00811333
Advisor: Professor Xin Zhang

Feb 2, 2026

Abstract

This paper presents the design and development of GeniusBank, a Software-as-a-Service (SaaS)-style banking application built using a Flask-based architecture. The system simulates core financial operations, including deposits, withdrawals, account upgrades, and user authentication, while emphasizing real-world information systems principles. In addition to the web-based platform, a graphical user interface (GUI) is implemented to facilitate user interaction with the system. The application incorporates secure authentication mechanisms, including third-party login integration via GitHub, demonstrating practical alternatives to traditional OAuth providers. The project highlights how modern fintech platforms integrate backend databases, authentication systems, and scalable service models, and further examines the relevance of GeniusBank within the broader context of SaaS and financial technology systems, including its practical applications, current limitations, and opportunities for future enhancement.

Keywords: SaaS, Cybersecurity, Flask, Fintech, Deposit, Withdrawals, user authentication, Github

1 Introduction

The rapid growth of financial technology (fintech) has significantly transformed how users interact with modern banking systems. Traditional banking infrastructures have increasingly shifted toward cloud-based and service-oriented architectures, enabling scalable, secure, and accessible financial services. This paper introduces GeniusBank, a SaaS-style banking application designed to replicate key functionalities of contemporary digital banking platforms.

The purpose of this project is to demonstrate the integration of core information systems concepts, including database management, authentication, transaction pro-

cessing, and client-server communication. Developed using Flask, SQLite, and GitHub-based OAuth authentication, GeniusBank reflects how real-world applications are structured and deployed in modern computing environments.

This paper analyzes the system architecture, core features, and security considerations of GeniusBank, while also examining its alignment with modern SaaS-based fintech platforms. Additionally, it discusses the challenges encountered during development and proposes future improvements to enhance scalability, security, and overall system usability.

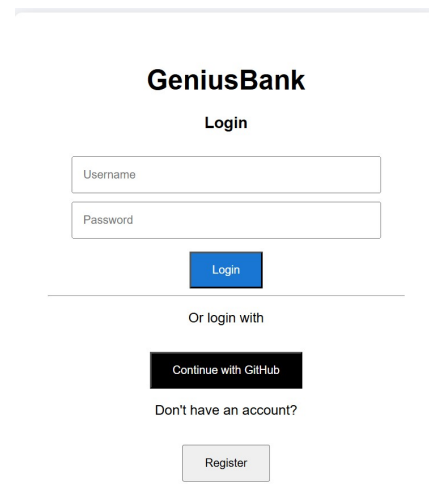


Figure 1: main website of what it looks like

2 Background and Problem Definition

Financial technology (fintech) represents the integration of technology into financial services to improve efficiency, accessibility, and overall user experience. Modern fin-

tech systems rely heavily on cloud computing, distributed architectures, and secure data management practices to support scalable and reliable operations. As digital banking continues to expand, there is an increasing demand for systems that not only provide core financial functionality but also demonstrate strong security, scalability, and usability principles.

One of the most prominent delivery models in this domain is Software-as-a-Service (SaaS), in which applications are centrally hosted and accessed over the internet. SaaS platforms eliminate the need for users to manage infrastructure, allowing services to scale dynamically based on demand while maintaining high availability. This model has become a foundation for many modern financial applications due to its flexibility and cost efficiency.

Despite these advancements, developing a system that effectively integrates secure authentication, transaction processing, and scalable architecture remains a complex challenge. Many learning environments and small-scale applications fail to fully demonstrate how these components interact within a unified system. This gap highlights the need for practical implementations that bridge theoretical concepts with real-world system design.

GeniusBank is designed to address this challenge by serving as a SaaS-style banking application that simulates core financial operations. The system integrates essential information systems components, including relational databases, authentication layers, and transaction logic, providing a cohesive platform that reflects how modern fintech systems are structured. By doing so, GeniusBank offers a practical demonstration of how secure, scalable, and user-focused financial applications can be developed.

Welcome, Vig270

Balance: \$99 | Plan: FREE

Welcome, Vig270

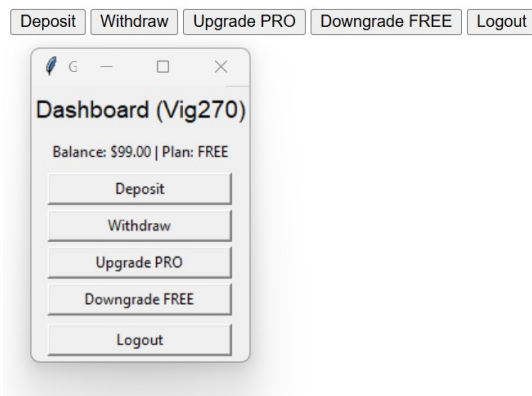


Figure 2: both login in github, this works in regular accounts too

3 Related Work

Modern digital banking systems and fintech applications have evolved significantly with the adoption of cloud computing and service-oriented architectures. Traditional online banking platforms provided by financial institutions such as mobile and web banking applications demonstrate core functionalities including user authentication, account management, and secure transaction processing. These systems rely on highly scalable infrastructures and strict security protocols to ensure reliability and data protection.

In the fintech industry, Software-as-a-Service (SaaS) platforms such as PayPal and Stripe illustrate how financial services can be delivered through cloud-based, API-driven architectures. These systems emphasize modular design, third-party authentication, and tiered service models, enabling scalable and flexible financial operations across large user bases.

From a software engineering perspective, many academic and learning-based projects implement simplified banking or CRUD systems using frameworks such as Flask or Django. While these systems effectively demonstrate backend development and database integration, they often lack production-level features such as secure authentication mechanisms, service differentiation, and scalable architecture design.

GeniusBank extends these foundational ideas by integrating secure password hashing with salting, GitHub OAuth-based authentication, a tiered Free and Pro account model, and a modular client-server architecture. This allows the system to bridge the gap between basic educational implementations and real-world SaaS-based fintech platforms, providing a more comprehensive demonstration of modern information systems principles.

4 Motivation

The motivation for developing GeniusBank stems from prior experience with a cybersecurity-focused banking application, which highlighted the importance of secure system design and user authentication. Building upon this foundation, the project was extended to incorporate Software-as-a-Service (SaaS) principles and cloud-oriented architecture, reflecting a growing interest in scalable and distributed systems. This progression enables a deeper exploration of how traditional security-focused applications can evolve into modern, service-oriented platforms. As cloud computing continues to play a central role in fintech, the development of GeniusBank provides an opportunity to examine the integration of security, scalability, and usability within a unified system.

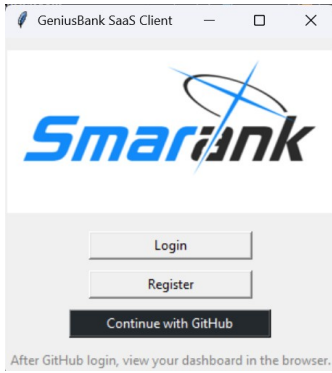


Figure 3: intro to dashboard visualization

5 System Overview

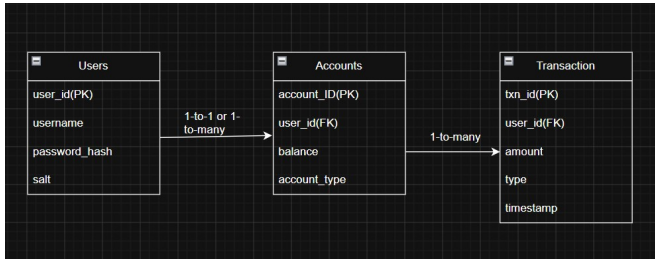


Figure 4: ER Diagram

The GeniusBank system is designed using a client-server architecture, where the backend is implemented using the Flask web framework and data is managed through a SQLite relational database. This architecture reflects a lightweight yet effective approach commonly used in small-scale SaaS applications and prototyping environments.

At the core of the system is the Flask backend, which handles HTTP requests, processes business logic, and communicates with the database. The application defines multiple API routes that support user authentication, financial transactions, and account management. These routes act as the interface between the client and server, ensuring that user actions are processed securely and efficiently.

The database layer is implemented using SQLite, which stores user-related data such as user identifiers, usernames, password hashes, salts, account balances, and account types. By utilizing a relational schema, the system maintains data consistency and enables efficient querying. For example, during authentication, stored password hashes and salts are retrieved and used to validate user credentials.

The system also incorporates an authentication module that manages user sessions and integrates GitHub OAuth for third-party login. This ensures that only authenticated users can access protected features such as deposits, withdrawals, and account upgrades. Session management

plays a critical role in maintaining user state across multiple interactions, preventing unauthorized access to sensitive operations.

From a structural perspective, the application is organized into modular components, including server-side logic (Flask application and database interaction), an authentication layer, and a client interface. This modular design improves maintainability, scalability, and separation of concerns, aligning with best practices in modern software engineering.

5.1 API Endpoints and User Interaction

GeniusBank provides a set of API endpoints that are directly linked to user interface actions within both the web-based platform and the graphical user interface (GUI) implementation. Each user interaction, such as clicking a button or submitting a form, triggers a corresponding backend route that processes the request and returns a response.

The `/signup` endpoint allows new users to create an account by providing a username and password. Alternatively, users may register through GitHub OAuth, where first-time authentication results in automatic account creation. The `/login` endpoint supports both traditional credential-based authentication and GitHub-based login `/Continue With Github`, enabling flexible access to the system.

Financial operations are handled through the `/deposit` and `/withdraw` endpoints. The deposit functionality allows users to add funds to their account, with restrictions based on account type. Free-tier users are limited to smaller deposit amounts, while Pro users have fewer restrictions. The withdrawal functionality enables users to remove funds, with a transaction fee applied to Free-tier accounts.

Account management features are implemented through the `/upgrade to PRO` and `/downgrade to FREE` endpoints. Upgrading to a Pro account removes deposit limits and transaction fees, while downgrading reintroduces these constraints. The `/signout` endpoint terminates the active session and returns the user to the main interface.

In addition to the web-based platform, GeniusBank includes a GUI-based implementation that interacts with the same backend system. This provides an alternative interface for users while maintaining consistent functionality and shared business logic across both environments.

Overall, the GeniusBank architecture demonstrates how a fintech-inspired SaaS application can be constructed using a service-oriented approach, integrating backend processing, secure authentication, and persistent data storage into a cohesive and maintainable system.

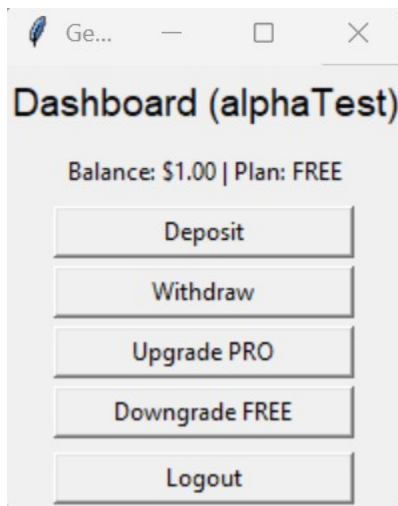


Figure 5: GUI Dashboard Interface

6 Diagrams and Models

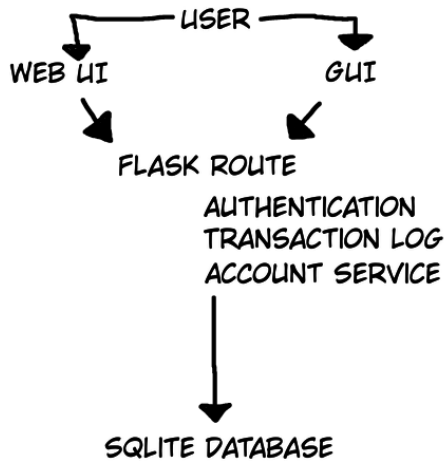


Figure 6: High-Level Architecture of GeniusBank

This is a diagram of how user can choose between the app version and web interface to login via github or their original login/password. It uses flask to make authentication, transaction and gives service. It will then record and save password in a hash while also saving it in a database.

7 Core Features and Functionality

GeniusBank implements a set of core features that simulate the behavior of a modern digital banking platform. These features demonstrate key information systems concepts, including authentication, transaction processing, data management, and service-level differentiation within a SaaS-style architecture.

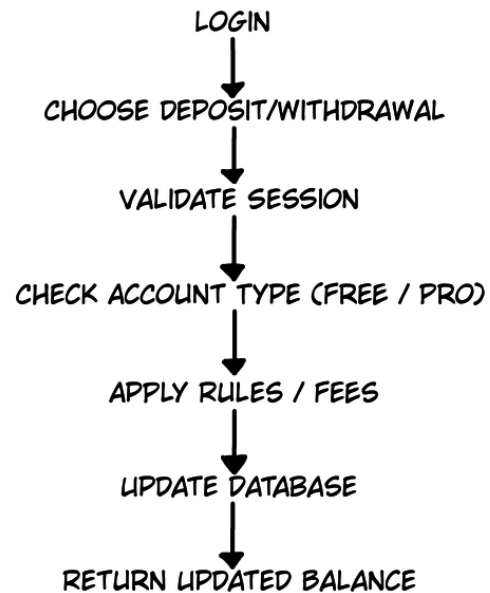


Figure 7: Transaction Processing Workflow

A primary feature of the system is user authentication. Users securely access the platform using credentials that are verified against stored password hashes and salts. This approach ensures that sensitive information is never stored in plain text and significantly reduces the risk of unauthorized access. In addition, GitHub OAuth integration provides an alternative authentication method, reflecting industry-standard practices for third-party login systems.

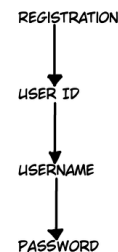


Figure 8: register way of making the account, the user id must be unique

The system also includes account management capabilities that allow users to interact with their financial data after authentication. Each user account is uniquely maintained within the database, ensuring data isolation and integrity. Once logged in, users can view account details and perform authorized financial operations.

Transaction processing is handled through deposit and withdrawal functionality. These operations are executed

via backend routes that validate active user sessions, enforce business rules, and update account balances accordingly. For example, withdrawal operations include validation checks to prevent users from exceeding their available balance, ensuring consistency and preventing invalid transactions.

GeniusBank further implements a tiered service model that distinguishes between Free and Pro users. This SaaS-based structure introduces differentiated functionality, where Pro users benefit from fewer restrictions and reduced transaction limitations. Account upgrades are processed through a dedicated system route that updates the user's account status in the database.

Overall, these features demonstrate how a fintech-inspired system manages user interactions, enforces business logic, and maintains secure and consistent data flow. The design emphasizes backend validation, modular functionality, and user-centric system behavior, reflecting principles commonly used in real-world financial applications.

8 Security and Data Management

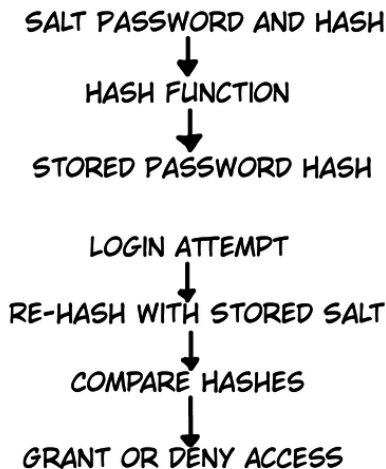


Figure 9: Password Hashing and Salt Verification Process

Security and data management are critical components of any financial system, and GeniusBank incorporates multiple mechanisms to ensure the protection of user information and system integrity. Given the sensitive nature of financial data, the system emphasizes secure authentication, controlled access, and reliable transaction processing.

A key security mechanism implemented in GeniusBank is password hashing combined with salting. Instead of storing passwords in plain text, the system stores a cryptographic hash of each password along with a unique salt value. The salt is a randomly generated sequence of bytes that is appended to the password before hashing, ensuring

that identical passwords produce different hash outputs. During authentication, the entered password is combined with the stored salt and re-hashed for comparison against the stored hash. This significantly improves security by protecting against rainbow table attacks and reducing the impact of database exposure.

Session management is another important aspect of system security. Once a user successfully logs in, a session is created to maintain authentication state across multiple requests. This ensures that only authenticated users can access protected routes such as deposits, withdrawals, and account upgrades. The system verifies session validity before processing any sensitive operation, preventing unauthorized access.

Data integrity is enforced through controlled database operations within the backend. All financial transactions are validated before execution to ensure correctness and consistency. For example, withdrawal operations include checks to prevent users from exceeding their available balance, while deposit operations enforce input validation to avoid invalid or malformed data entries.

Additionally, the system reduces security risks by restricting direct client access to the database. All database interactions are handled exclusively through backend logic, minimizing exposure to injection-based attacks and ensuring a clear separation between the user interface and data storage layers.

Although GeniusBank is implemented as a prototype, it reflects several real-world security practices commonly used in financial systems, including hashed credential storage, salted passwords, session-based authentication, and backend validation of critical operations. Together, these mechanisms contribute to a more secure and reliable application design.

9 SaaS and Real-World Relevance

GeniusBank reflects key principles of Software-as-a-Service (SaaS) and modern financial technology systems. SaaS applications are typically centrally hosted, accessible over the internet, and designed to scale dynamically to support multiple users. In this architecture, the backend is responsible for processing logic and data management, while users interact through a lightweight client interface. GeniusBank adopts this client-server model, where Flask handles backend processing and SQLite manages persistent data storage.

A fundamental aspect of SaaS systems is multi-user support with secure and isolated data management. In GeniusBank, each user account is uniquely stored and managed within the database, ensuring that financial data remains private and segregated between users. This mirrors real-world fintech systems, where multi-tenancy and strict data isolation are essential for security and regulatory compliance.

The system also incorporates a tiered service structure,

distinguishing between Free and Pro accounts. This design reflects common SaaS business models in which users subscribe to different service levels to access additional features or reduced restrictions. Such models are widely used in fintech applications to balance accessibility for new users with monetization strategies that support platform sustainability.

Although GeniusBank is implemented in a local development environment, its architecture aligns with cloud-native design principles. The separation between frontend interaction, backend processing, and persistent storage reflects how modern applications are deployed in cloud platforms such as AWS or Azure. In a production setting, this system could be extended using load balancing, containerization, and distributed databases to support higher scalability and availability.

Overall, GeniusBank demonstrates how SaaS principles can be applied within a fintech context to build scalable, user-centric systems. It serves as a practical implementation of service-oriented architecture, illustrating how secure data management, modular backend design, and tiered service delivery can be integrated into a cohesive financial application.

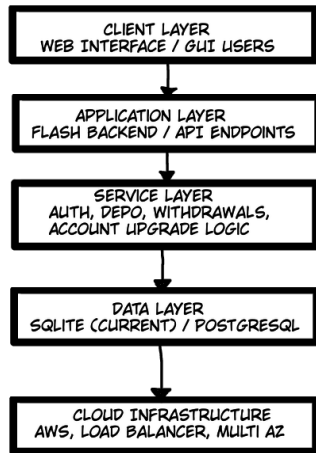


Figure 10: Conceptual SaaS Deployment Model

10 Challenges and Limitations

The development of GeniusBank involved several technical challenges that highlight the complexities of designing and implementing a secure and reliable financial system. Google OAuth authentication was considered as part of the system design; however, GitHub OAuth was ultimately implemented as an alternative third-party authentication provider due to external service configuration constraints and API setup requirements. This decision still allows the system to demonstrate industry-standard federated authentication while maintaining a fully functional login system. If funding was involved to fully complete this project, using a Virtual Machine (VM) such

as AWS or Azure would benefit this. And this would enhance the SaaS-like a fully SaaS.

Another significant challenge was maintaining system stability while introducing new features. As functionality such as account upgrades and transaction processing was added, previously working components occasionally required modification or debugging. This reflects a common challenge in software engineering, where system complexity increases as new features are integrated. It also highlights the importance of modular design and comprehensive testing to ensure that updates do not introduce unintended side effects.

Database limitations also constrained system design decisions. The project utilizes SQLite as its primary database, which is well-suited for lightweight applications and prototyping but limited in terms of scalability and concurrent user handling. In contrast, production-level systems typically rely on database technologies such as PostgreSQL or MySQL, which provide stronger concurrency control, scalability, and reliability for high-traffic environments.

In addition, GeniusBank does not yet implement several advanced features commonly found in production fintech systems. These include real-time fraud detection, detailed transaction logging, and integration with external financial APIs. While basic input validation and error handling are implemented, these mechanisms could be further strengthened to improve system robustness and security. Using examples from Capital One or Chase would give a strong understanding of a prototype virtual banking app.

In the end, these challenges demonstrate that although GeniusBank successfully implements core fintech and SaaS concepts, it remains a prototype system. Addressing these limitations would be necessary to evolve the application into a fully production-ready financial platform with enterprise-level reliability and scalability.

11 Findings

The implementation of GeniusBank demonstrated that core fintech functionalities such as authentication, transaction processing, and account management can be effectively integrated using a lightweight Flask-based architecture. The system successfully supported user registration, login, and session management, ensuring that only authenticated users could access sensitive operations.

One of the key findings is that password hashing combined with salting provides a reliable mechanism for securing user credentials, significantly reducing the risk of plaintext password exposure. Additionally, session-based authentication proved essential in maintaining secure and consistent user access across multiple transactions.



Figure 11: salt password visualized in notepad, copied and pasted from terminal

The transaction system, including deposits and withdrawals, operated correctly with real-time updates to the SQLite database. Validation checks successfully prevented invalid operations such as overdrafts, demonstrating the importance of backend enforcement of business rules.

It was also observed that separating system components into modular layers (authentication, transaction logic, and database management) improved maintainability and reduced system complexity. The integration of GitHub OAuth further showed that third-party authentication can enhance usability while reducing the need for custom login systems.

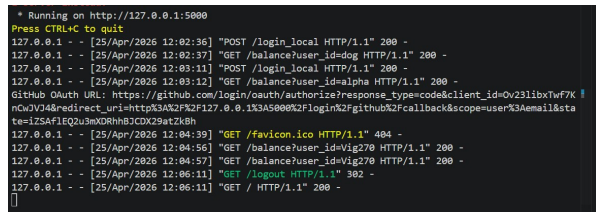


Figure 12: comes with key and hashing

However, the findings also highlighted limitations in scalability, as SQLite is not suited for high-concurrency environments. This reinforces the need for more robust database systems such as PostgreSQL in production-level deployments.

Overall, the system effectively demonstrates how SaaS-inspired fintech applications can be implemented at a prototype level, bridging theoretical concepts with practical system design.

12 Future Improvements

There are several opportunities to enhance GeniusBank and evolve it into a more production-oriented fintech application. A primary improvement would be migrating the system from a local deployment environment to a cloud-based infrastructure. Deploying the application on platforms such as Amazon Web Services (AWS) would enable improved scalability, high availability, and fault tolerance. Services such as load balancing, auto-scaling,

and multi-availability zone deployment could significantly enhance system reliability under increased user demand.

Another key enhancement involves upgrading the database layer. Transitioning from SQLite to a more robust relational database system such as PostgreSQL would improve concurrency handling, data integrity, and scalability. This would be essential for supporting a larger user base and more frequent transaction workloads in a real-world environment.

Security improvements could further strengthen the system by introducing multi-factor authentication (MFA), encrypted communication using HTTPS, and enhanced auditing through comprehensive transaction logging. These additions would align the system more closely with industry standards used in modern financial applications.

From a functionality perspective, integrating external financial APIs such as payment gateways or banking services would allow GeniusBank to simulate real-world financial interactions more accurately. Additionally, incorporating analytics or machine learning-based features for fraud detection or transaction anomaly monitoring could extend the system's intelligence and practical value.

Finally, the user interface could be modernized through a more responsive web framework or mobile application interface, improving usability and accessibility. These enhancements collectively align with cloud computing and SaaS principles, many of which are reflected in AWS Cloud Practitioner (CLF-C02) concepts, particularly in areas such as scalable architecture, infrastructure management, and cloud service deployment models.

Overall, these improvements would transition GeniusBank from a prototype system into a scalable, secure, and cloud-ready fintech platform.

13 Conclusion

The GeniusBank system demonstrates the practical application of information systems concepts within the context of a fintech-inspired SaaS platform. Through its design and implementation, the project illustrates how backend development, database management, authentication mechanisms, and transaction processing can be integrated to create a functional digital banking system.

The system emphasizes critical principles such as security, data integrity, and user management, incorporating mechanisms such as password hashing, session-based authentication, and transaction validation that reflect practices used in real-world applications. In doing so, GeniusBank provides a practical example of how theoretical concepts in software engineering and information systems can be translated into a working solution.

Although the system has limitations, it establishes a strong foundation for understanding scalable and service-oriented architecture design. The challenges encountered during development further highlight the complexities in-

volved in building secure and reliable software systems.

Overall, GeniusBank serves not only as a prototype banking platform but also as a demonstration of how fintech, SaaS principles, and secure system design can be combined within a cohesive application. With continued enhancement, the project has the potential to evolve into a more advanced and production-ready financial system.

References

- [1] Flask Documentation.
<https://flask.palletsprojects.com/>
- [2] OAuth 2.0 Authorization Framework.
<https://oauth.net/2/>
- [3] Armbrust, M. et al. (2010). A View of Cloud Computing.
- [4] Cloud Knowledge <https://www.coursera.org/articles/aws-cloud-practitioner>